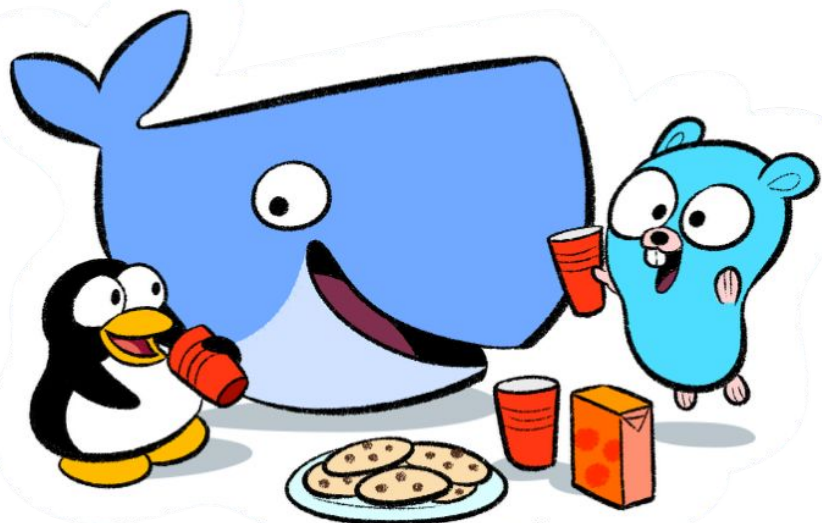


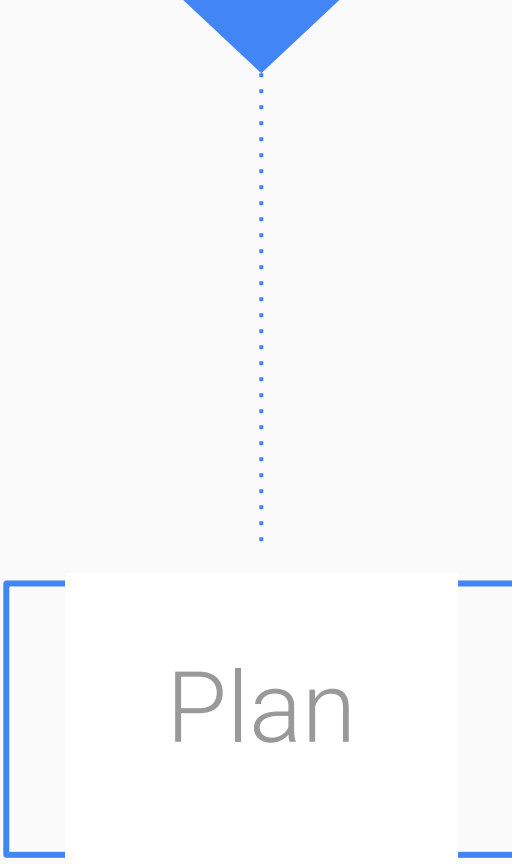


Architectures microservices et conteneurs

Fabrice Jammes, Karim Ammous, <https://k8s-school.fr>

This work is licensed under a [Creative Commons Attribution 4.0 International License](https://creativecommons.org/licenses/by/4.0/) . CC-BY





Plan

1

Le constat

2

Les conteneurs: exemple de Docker

3

Les Microservices: c'est quoi

4

L' Orchestration

Le constat

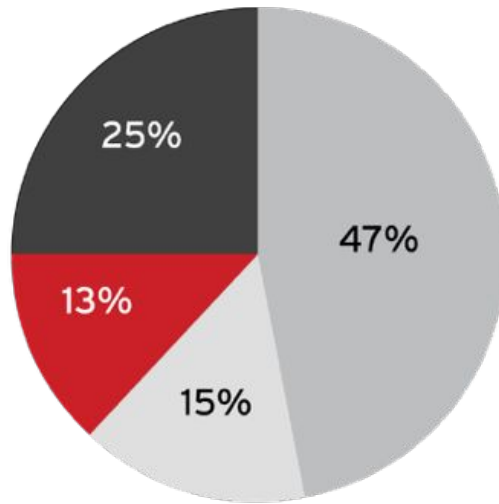
- Les **géants du web** ont changé leurs méthodes de développement et de déploiement depuis plusieurs années pour être plus efficace.
- Les **petites entreprises innovantes** utilisent maintenant les mêmes méthodes.
- Docker, Devops, Orchestration, Microservices, ...
- **Et nous ?**

Accélération

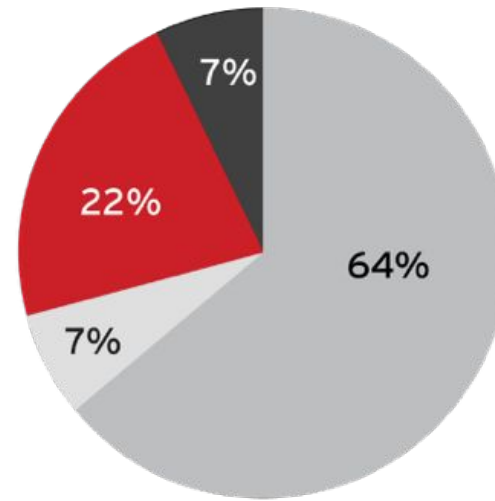
Datacenter	Virtualisation	Docker
		
Déploiement dans le mois	Déploiement dans la minute	Déploiement dans la seconde
Pendant des années	Pendant des mois	Pendant quelques heures/minutes
Développement en cascade	Agile	DevOps

Plus rapide et de meilleure qualité

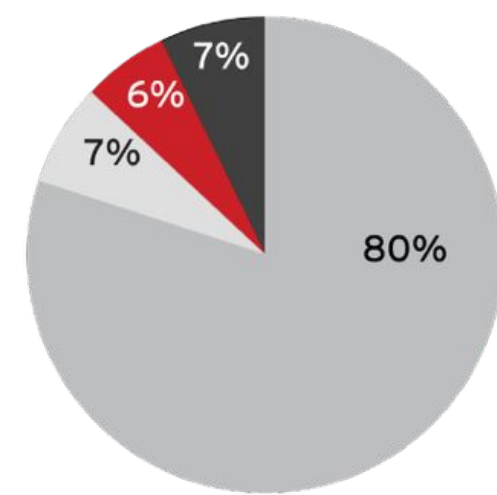
Développement
en cascade



Agile



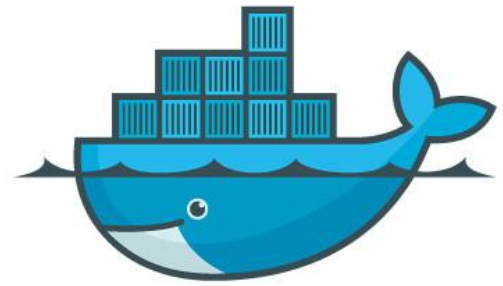
DevOps



Plus de temps consacré au
développement

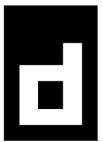
Devops: the big picture

QUI ?	QUOI ?	COMMENT ?
Devops	Application Cloud + Microservices	Orchestration de conteneurs



docker

Les conteneurs

container 

 Rocket



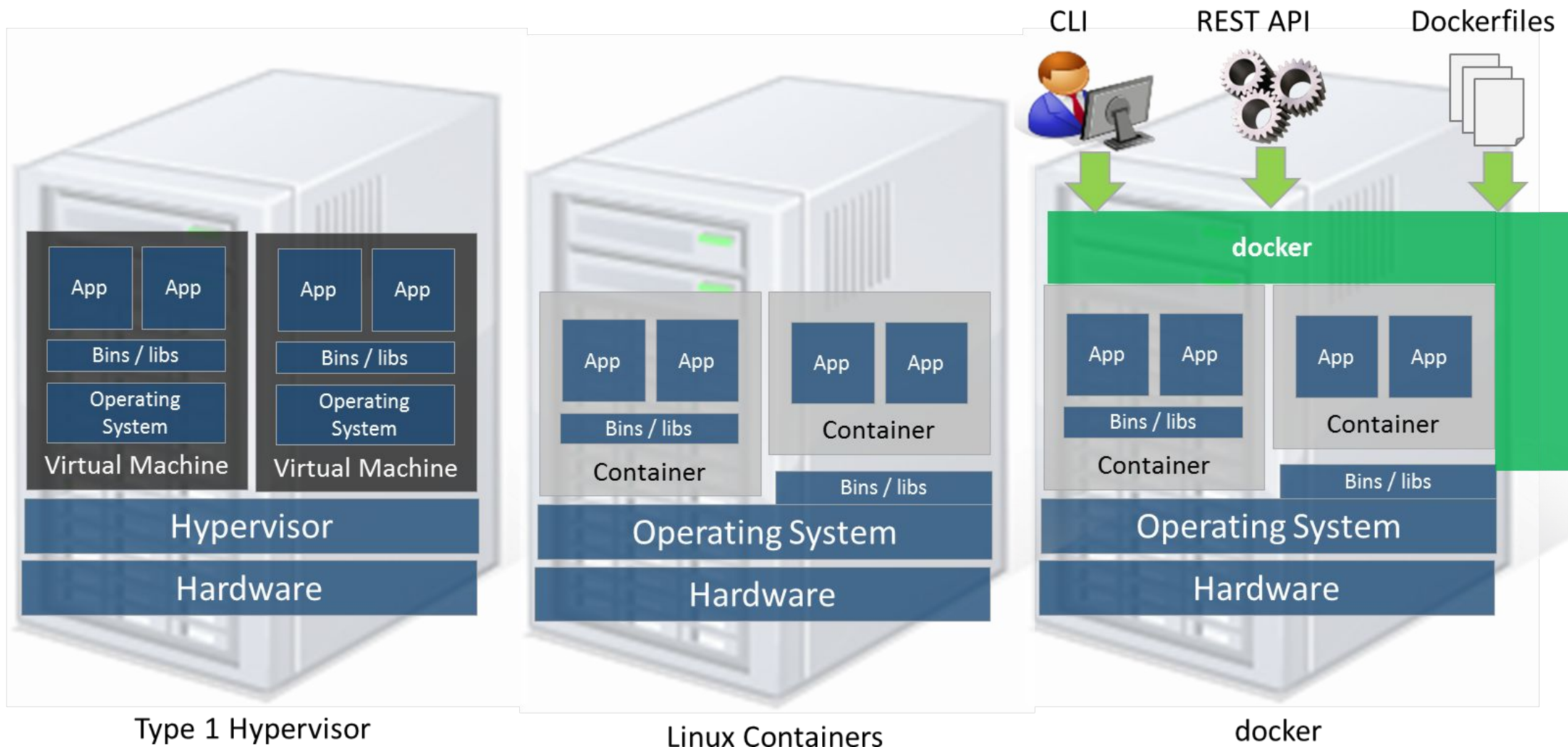
CRI-O: OCI-based
Kubernetes Runtime



podman



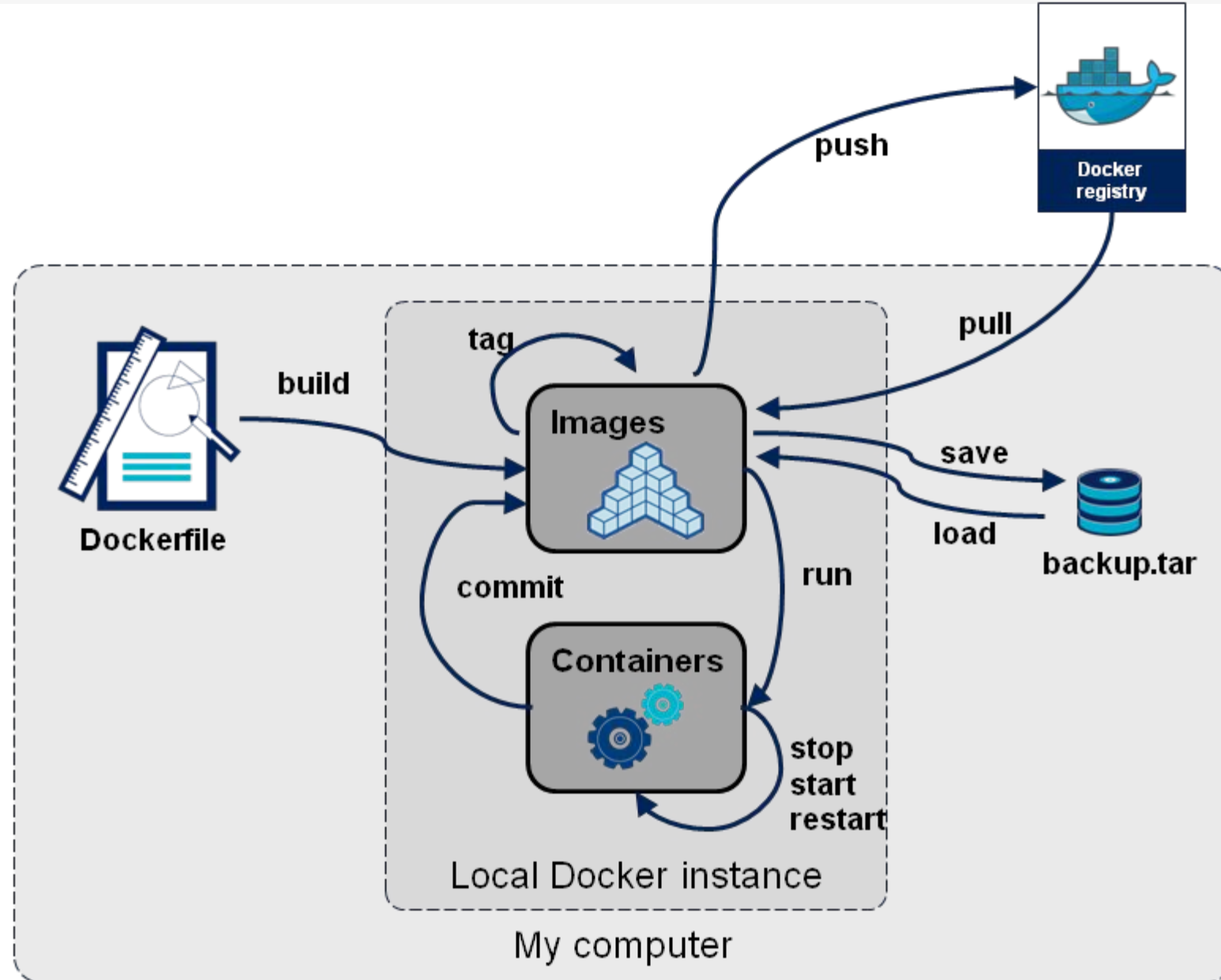
Hyperviseur vs Docker



Les outils Docker libres

- DOCKER MOTEUR/CLIENT: Moteur et le client pour utiliser docker en cli.
- DOCKER COMPOSE: Lancer des applications multi-containers.
- DOCKER SWARM: Gerer de containers Docker dans un cluster.
- DOCKER REGISTRY: Application de gestion des images locales.
- NOTARY: Signature des images par le fournisseur et vérification de l'intégrité par le client.
- LIBNETWORK: abstraction réseau permettant la communication et l'isolation entre les containers, même sur des nœuds différents.

Docker Moteur/Client



Pourquoi utilise-t-on Docker?

- Distribution des applications facilitée.
- Comportement identique des applications en Dev/Qualif/Prod.
- Déploiement, lancement et arrêt rapide.
- Linux et Windows
- Permet de reconstruire un container à partir d'un simple fichier Dockerfiles.
- Gestion des containers avec peu d'outils, identique sur toutes les plateformes.
- Des API disponibles pour piloter l'ensemble depuis d'autres applications.

The twelve factors

<http://12factor.net>

The twelve factors

I. Codebase

One codebase tracked in revision control, many deploys

II. Dependencies

Explicitly declare and isolate dependencies

III. Config

Store config in the environment

IV. Backing services

Treat backing services as attached resources

V. Build, release, run

Strictly separate build and run stages

VI. Processes

Execute the app as one or more stateless processes

VII. Port binding

Export services via port binding

VIII. Concurrency

Scale out via the process model

IX. Disposability

Maximize robustness with fast startup and graceful shutdown

X. Dev/prod parity

Keep development, staging, and production as similar as possible

XI. Logs

Treat logs as event streams

XII. Admin processes

Run admin/management tasks as one-off processes

- Tout code doit être géré par un logiciel de suivi de version (git, mercurial, ...).
- Une application = code source

Dependencies

- Toutes les dépendances doivent être clairement précisées.
- Le système cible n'est pas censé contenir de programme pré-installé.
- Pas de dépendances implicites.

- Est considéré comme configuration, tout ce qui diffère d'un environnement à l'autre (dev, qualif, prod, autre site).
- Tout élément de configuration doit être passé par l'environnement.
- Il ne doit y avoir absolument aucune référence à la configuration dans le code.

Backing Services

- Un backing service est une ressource externe au conteneur (base mysql, smtp, activemq, memcache, ...).
- L'accès à ces ressources doit être passé en paramètre.
- Pas de distinction entre les services locaux et distants.

- On recrée l'application et l'environnement avant tout déploiement d'une nouvelle version.
- Aucune modification n'est apportée sur l'application déployée.
- Chaque version déployée a un numéro de version unique (timestamp, numero de commit, ...).

Processes

- L'application est exécutée dans l'environnement d'exécution en tant qu'un ou plusieurs processus.
- Toutes les données doivent être stockées dans une ressource externe (base de données).
- Les variables de sessions utilisateurs ne doivent jamais être stockées localement.

Port Binding

- L'application fournit un service qui écoute sur un port.

- Chaque application peut être mise à l'échelle. Les conteneurs peuvent être lancés x fois pour répartir la charge.
- Le programme dans le conteneur ne doit pas être lancé en tâche de fond.
- L'arrêt du programme entraîne l'arrêt du conteneur.

- Le conteneur doit être jetable.
- Il doit donc pouvoir être lancé très rapidement.
- Un arrêt intempestif ne doit pas compromettre les données.

Dev/prod parity

- Le développeur doit pouvoir déployer rapidement le code qu'il vient de finir d'écrire.
- Le développeur doit être plus proche du déploiement (DevOps/SRE).
- Maintenir le développement et la production aussi semblables que possible en utilisant les mêmes outils.
- Éviter de prendre des backends différents en prod et en dev (ex: base de données, ...) pour limiter les surprises en production.

- Les applications doivent externaliser leurs journaux pour la visualisation et l'archivage à long terme (ELK, Spunk, rsyslog ...).
- Les journaux peuvent s'afficher dans la sortie standard de l'application, mais pas dans un fichier du conteneur.

- Les commandes d'administration doivent s'exécuter dans un environnement identique aux autres processus d'exploitation.
- Même conteneur, mêmes variables d'environnement, mais en mode interactif.

12-FACTORS C'EST BIEN MAIS...

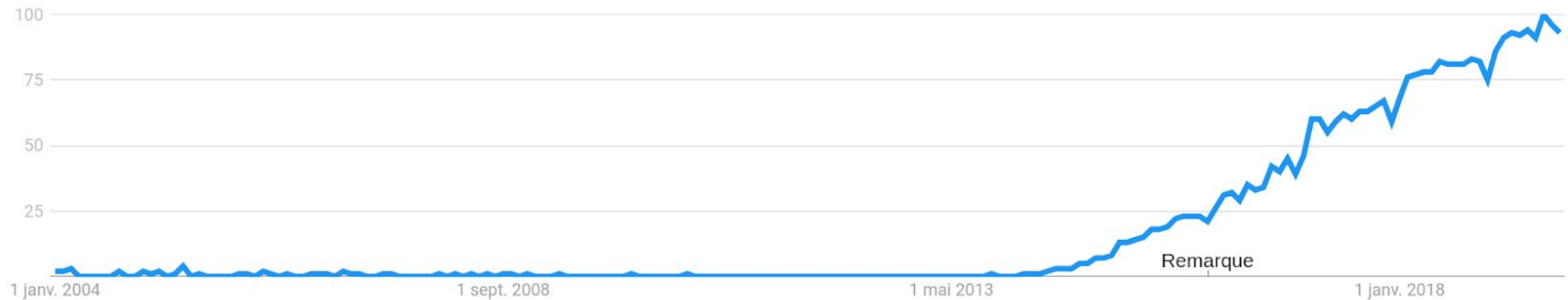
...Comment les mettre en oeuvre de façon efficace.

Microservices

Juste un buzz?

- Micro-Web-Services en 2005
- Définition Microservices en 2011
- Microservices = SOA mais seulement les bonnes parties.

Évolution de l'intérêt pour cette recherche ?



Intérêt relatif par rapport à d'autres outils

● microservices Terme de recherche	● kubernetes Terme de recherche	● docker Terme de recherche	+ Ajouter une comparaison
---	--	--	----------------------------------

Dans tous les pays ▼

De 2004 à ce jour ▼

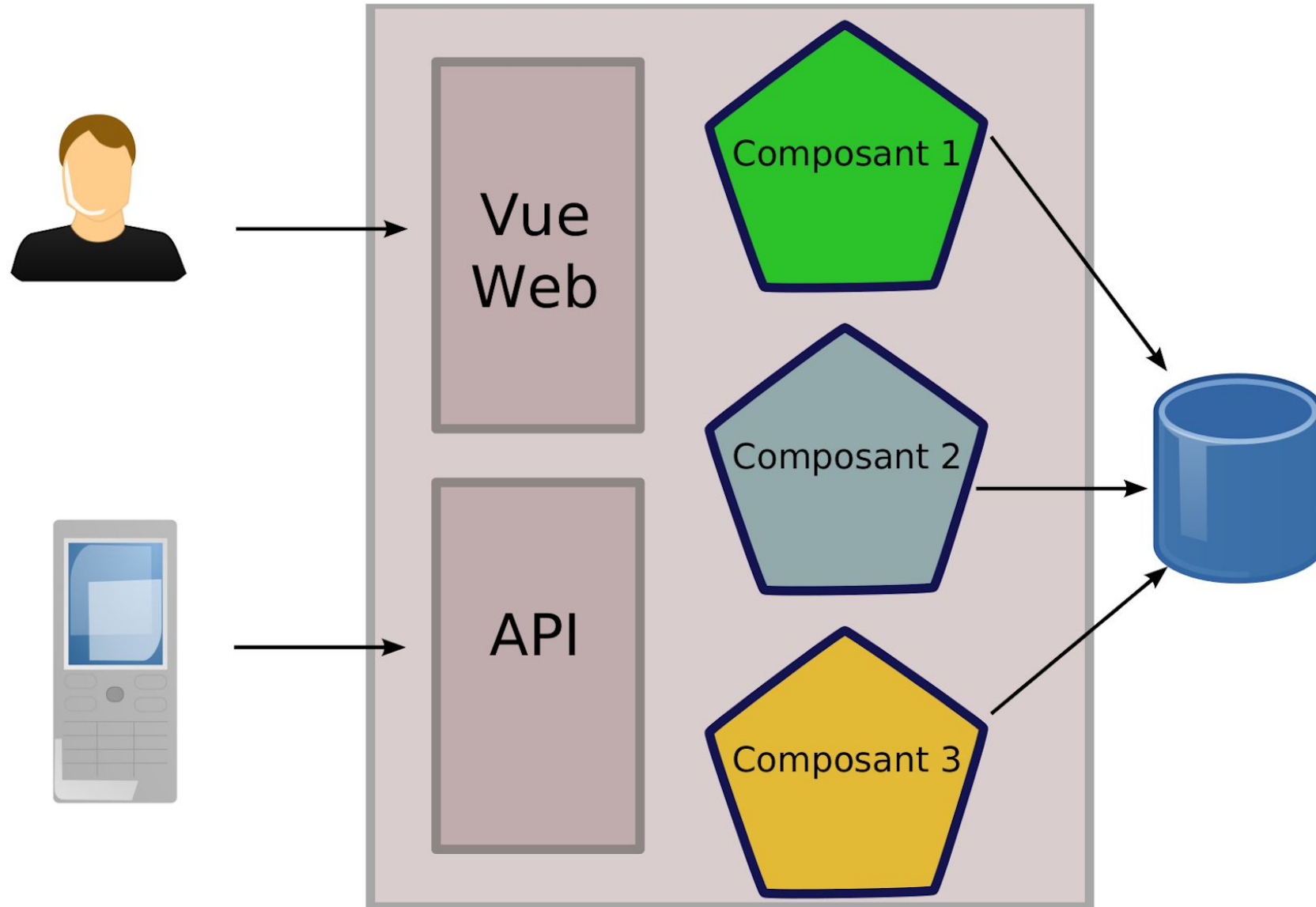
Toutes catégories ▼

Recherche sur le Web ▼

Évolution de l'intérêt pour cette recherche ?



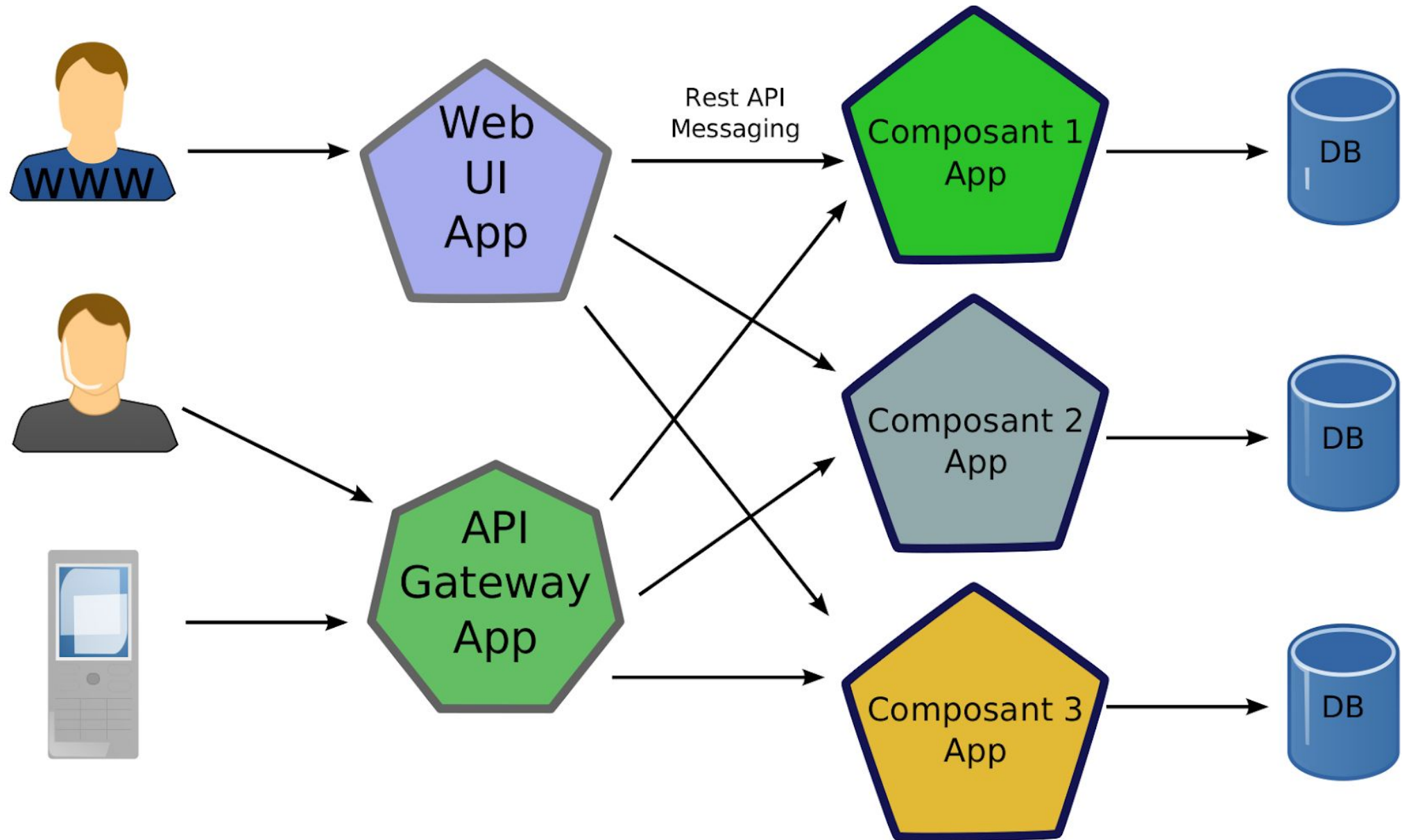
Les applications monolithiques



Les applications monolithiques

- Applications N-tiers.
- IHM, core, accès aux BDD.
- Chaque modification nécessite de redéployer la
- totalité de l'application.
- Chaque modification nécessite de retester l'ensemble
- de l'application.
- Difficile de garder au fil du temps une bonne structure
- modulaire.
- Mise à l'échelle coûteuse.
- Manque de diversité technologique.
- Pas facile de changer un composant.

Les applications Microservices



ÉLASTIQUE

Un Microservice doit pouvoir être déployé un nombre de fois qui varie en fonction de la demande, et ce, indépendamment des autres services dans la même application.

RÉSILENT

Un Microservice doit échouer sans affecter d'autres services dans la même application.

API

Les Microservices doivent avoir une API stable, cohérente et bien documentée. <https://openapis.org/>

MINIMAL MAIS COMPLET

Un Microservice doit être le plus petit possible mais pas plus petit. Il doit offrir une fonction complète avec des dépendances minimales avec les autres services.

Avantages

- Facilite le travail en équipe.
- Augmente la qualité globale des applications.
- Permet d'utiliser le meilleur langage de programmation en fonction de la problématique.
- Permet d'automatiser les tests, la qualification et le déploiement en production.
- Chaque Microservice peut être mis à l'échelle indépendamment et peut être optimisé sans affecter le reste de l'application.
- Augmentation de la robustesse de l'architecture, de la tolérance aux pannes.

Inconvénients

- Nécessite un bon niveau d'expertise DevOps.
- Gestion décentralisée des données.
- Application polyglotte.
- Augmentation du trafic réseau.
- Coût initial plus élevé.
- Sécurité des communications entre les Applis.

Tous ces points peuvent être résolus par l'utilisation d'outils adéquats et d'un peu d'huile de coude.

La motivation qui pousse à utiliser les Microservices

- Arrivée sur le marché d'outils permettant le déploiement des applications Microservices avec plus de facilité.
- Large adoption des solutions d'infrastructure en tant que service (IaaS).
- Le passage des grosses sociétés du Web vers des architectures complètement Microservices.
- Frustration de ne pas obtenir le résultat souhaité avec une architecture monolithique.

Dans un avenir plus ou moins proche les applications monolithiques ne serviront qu'au prototypage.

Questions?

Merci!

[@fjammes](#) on Github



<https://k8s-school.fr>